

Documentación Técnica

Sistema de Procesamiento de Señales Biomédicas

Miguel Angel Toro Montealegre

CC 1077851975

Stefania Concha Cáceres

CC 1093592120

ECG & EMG — Pipeline con POO, NumPy, SciPy y Matplotlib

Taller 1 | Programación Orientada a Objetos | Informática 2

1. Descripción general del proyecto

El proyecto implementa un mini-sistema en Python con Programación Orientada a Objetos (POO) para cargar o generar señales biomédicas 1D (Electrocardiograma y Electromiograma), procesarlas mediante un pipeline de filtrado, detectar eventos fisiológicos y reportar métricas con sus respectivas figuras.

Los principios de POO aplicados son:

- Encapsulación: todos los atributos son privados con prefijo `_` y expuestos mediante `@property` con validación.
- Herencia y polimorfismo: `Processor` es la clase base abstracta; `BandpassFilter`, `NotchFilter` y `EventDetector` la heredan e implementan el método `process()`.
- Abstracción: el `Pipeline` no conoce los detalles internos de cada procesador, solo que todos responden a `process()`.

2. Estructura de archivos

Archivo	Descripción
<code>biosignal.py</code>	Clase <code>BioSignal</code> — representa la señal 1D con validación
<code>processor.py</code>	Clase abstracta <code>Processor</code> — interfaz base con <code>process()</code>
<code>filters.py</code>	<code>BandpassFilter</code> y <code>NotchFilter</code> — subclases de <code>Processor</code>
<code>event_detector.py</code>	<code>EventDetector</code> — detecta latidos (ECG) o activaciones (EMG)
<code>pipeline.py</code>	<code>Pipeline</code> — encadena procesadores en secuencia
<code>experiment_runner.py</code>	<code>ExperimentRunner</code> — orquesta experimentos, guarda figuras y CSV
<code>main.py</code>	Script ejecutable principal

El directorio de resultados se genera automáticamente al ejecutar `main.py`:

```
results/  
  figures/  ← imágenes PNG de cada experimento  
  metrics.csv ← bitácora de métricas
```

3. Librerías utilizadas

Tipo	Librería	Uso
Externa	<code>numpy</code>	Arreglos, operaciones matemáticas, generación de señales sintéticas
	<code>scipy</code>	<code>scipy.signal</code> : filtros Butterworth, notch, <code>find_peaks</code>

	matplotlib	Generación y guardado de figuras PNG
Estándar	abc	Clase abstracta ABC y decorador abstractmethod
	os	Creación de carpetas (makedirs) y rutas (path.join)
	csv	Escritura de la bitácora metrics.csv
	datetime	Timestamps de cada experimento
	typing	Type hints: List, Optional, tuple

4. Documentación de clases

4.1 BioSignal — biosignal.py

Representa una señal biomédica 1D (ECG o EMG) con su frecuencia de muestreo y metadatos básicos. Es el contenedor de datos que circula por todo el pipeline.

Tipo	Nombre	Descripción
Atributo	<code>_data : ndarray</code>	Arreglo NumPy 1D con las muestras. Validado: debe ser 1D y no vacío.
Atributo	<code>_fs : float</code>	Frecuencia de muestreo [Hz]. Debe ser número positivo.
Atributo	<code>_signal_type : str</code>	Tipo de señal: 'ECG' o 'EMG'. Validado contra lista permitida.
Atributo	<code>_name : str</code>	Nombre descriptivo opcional de la señal.
Método	<code>data, fs, signal_type</code>	@property con setter validado para cada atributo privado.
Método	<code>duration : float</code>	Propiedad derivada: $\text{len}(\text{data}) / \text{fs} \rightarrow$ duración en segundos.
Método	<code>time : ndarray</code>	Propiedad derivada: eje temporal de la señal en segundos.
Método	<code>copy()</code>	Devuelve una copia independiente del objeto BioSignal.

4.2 Processor — processor.py

Clase base abstracta (interfaz) que define la firma del método `process()`. Todas las subclases deben implementarlo — garantiza polimorfismo.

Tipo	Nombre	Descripción
Atributo	<code>_name : str</code>	Nombre descriptivo del procesador.

Atributo	<code>_params : dict</code>	Diccionario de parámetros de configuración.
Método	<code>process(signal) → BioSignal</code>	Método abstracto. Recibe una BioSignal y devuelve una BioSignal transformada.

4.3 BandpassFilter — filters.py

Filtro pasa-banda Butterworth. Hereda de Processor. Deja pasar solo las frecuencias dentro del rango [lowcut, highcut] y elimina el resto.

Tipo	Nombre	Descripción
Atributo	<code>_lowcut : float</code>	Frecuencia de corte inferior [Hz]. Debe ser positiva.
Atributo	<code>_highcut : float</code>	Frecuencia de corte superior [Hz]. Debe ser > lowcut.
Atributo	<code>_order : int</code>	Orden del filtro Butterworth (default 4). Entero positivo.
Método	<code>process(signal)</code>	Normaliza frecuencias a Nyquist, calcula coeficientes con <code>scipy.signal.butter()</code> y aplica <code>filtfilt()</code> .

Detalle de implementación:

```
nyq = signal.fs / 2.0
b, a = scipy_signal.butter(order, [lowcut/nyq, highcut/nyq], btype='band')
filtered = scipy_signal.filtfilt(b, a, signal.data)
```

filtfilt aplica el filtro en dos pasadas (ida y vuelta), eliminando el desfase temporal que introduciría `filter`.

4.4 NotchFilter — filters.py

Filtro notch (rechaza-banda estrecha). Hereda de Processor. Elimina una frecuencia específica sin afectar las bandas vecinas.

Tipo	Nombre	Descripción
Atributo	<code>_freq : float</code>	Frecuencia a eliminar [Hz] (default 50 Hz — red eléctrica Colombia/Europa).
Atributo	<code>_quality : float</code>	Factor Q. Mayor Q = banda más estrecha de eliminación (default 30).
Método	<code>process(signal)</code>	Calcula coeficientes con <code>scipy.signal.iirnotch()</code> y aplica <code>filtfilt()</code> .

```
b, a = scipy_signal.iirnotch(freq, quality, signal.fs)
```

```
filtered = scipy_signal.filtfilt(b, a, signal.data)
```

4.5 EventDetector — event_detector.py

Detector de eventos fisiológicos. Hereda de Processor. Internamente bifurca su lógica según el tipo de señal: detecta picos R en ECG y activaciones musculares en EMG.

Tipo	Nombre	Descripción
Atributo	<code>_threshold_factor :</code> <code>float</code>	Fracción del máximo de la señal para el umbral de detección ($0 < \text{factor} \leq 1$).
Atributo	<code>_min_distance_s :</code> <code>float</code>	Distancia mínima entre eventos detectados en segundos.
Atributo	<code>_events :</code> <code>list</code>	Lista de índices de los eventos detectados tras <code>process()</code> .
Atributo	<code>_metrics :</code> <code>dict</code>	Diccionario con métricas calculadas (HR, RMS, etc.) tras <code>process()</code> .
Método	<code>process(signal)</code>	Llama a <code>_process_ecg()</code> o <code>_process_emg()</code> según <code>signal.signal_type</code> .
Método	<code>_process_ecg()</code>	Usa <code>find_peaks()</code> para detectar picos R. Calcula ritmo cardíaco: $60 / \text{RR_medio}$.
Método	<code>_process_emg()</code>	Calcula RMS de la señal. Detecta flancos de subida sobre umbral como activaciones.

4.6 Pipeline — pipeline.py

Cadena de procesamiento que aplica una lista de Processors en orden sobre una BioSignal. Guarda las señales intermedias de cada etapa para graficarlas luego.

Tipo	Nombre	Descripción
Atributo	<code>_processors :</code> <code>List[Processor]</code>	Lista de procesadores en orden de ejecución.
Atributo	<code>_intermediate_signals :</code> <code>List[BioSignal]</code>	Señal original + resultado de cada paso tras <code>run()</code> .
Método	<code>add(processor) →</code> <code>Pipeline</code>	Añade un procesador al final de la lista. Devuelve <code>self</code> para encadenamiento.
Método	<code>run(signal) → BioSignal</code>	Aplica cada procesador en orden. La salida de uno es la entrada del siguiente.
Método	<code>summary() → str</code>	Devuelve una descripción legible del pipeline y sus pasos.

Lógica central de `run()`:

```
current = signal
```

```

for proc in self._processors:
    current = proc.process(current)
    self._intermediate_signals.append(current)

```

4.7 ExperimentRunner — experiment_runner.py

Orquestador de experimentos. Ejecuta el pipeline sobre una señal, genera las figuras PNG por etapas y acumula los resultados en la bitácora CSV.

Tipo	Nombre	Descripción
Atributo	<code>_output_dir : str</code>	Ruta base donde se guardan resultados (default 'results/').
Atributo	<code>_figures_dir : str</code>	Ruta de la subcarpeta de figuras ('results/figures/').
Atributo	<code>_results : list</code>	Lista de diccionarios con los registros de cada experimento.
Atributo	<code>_run_count : int</code>	Contador de experimentos ejecutados.
Método	<code>run(pipeline, signal, detector)</code>	Ejecuta el pipeline, lee métricas del detector, llama a <code>_save_figure()</code> y registra el resultado.
Método	<code>_save_figure(...)</code>	Genera figura con subplots por etapa usando matplotlib. Guarda PNG en results/figures/.
Método	<code>save_metrics_csv(filename)</code>	Escribe todos los registros acumulados en un archivo CSV con codificación UTF-8.

5. Filtros y justificación

Se aplican dos filtros en cada pipeline, en el siguiente orden:

Señal	Filtro	Parámetros	Justificación
ECG	NotchFilter	50 Hz, Q=30	Elimina interferencia de red eléctrica
ECG	BandpassFilter	0.5 – 40 Hz	Aisla banda cardíaca, elimina deriva y ruido muscular
EMG	NotchFilter	50 Hz, Q=30	Elimina interferencia de red eléctrica
EMG	BandpassFilter	20 – 450 Hz	Aisla activaciones musculares

Por qué NotchFilter primero: la interferencia de 50 Hz puede saturar el filtro pasa-banda si se aplica después, ya que su amplitud supera en varios órdenes de magnitud a la señal fisiológica.

Por qué filtfilt: a diferencia de `lfilter` (una sola pasada), `filtfilt` aplica el filtro dos veces en direcciones opuestas, eliminando el desfase de fase que correría los eventos en el tiempo e invalidaría el cálculo de métricas.

6. Métricas calculadas

Señal	Campo CSV	Descripción
ECG	mean_heart_rate_bpm	Ritmo cardíaco medio = 60 / distancia media entre picos R [bpm]
ECG	num_beats	Número de latidos detectados en la señal
ECG	mean_rr_interval_s	Intervalo R-R promedio en segundos
EMG	rms	Raíz cuadrática media de la señal
EMG	num_activations	Número de inicios de activación muscular detectados
EMG	threshold_used	Umbral de amplitud utilizado para la detección

Fórmula del ritmo cardíaco medio (ECG):

```
HR [bpm] = 60 / mean(RR_intervals)
donde RR_interval[i] = (pico[i+1] - pico[i]) / fs
```

Fórmula del RMS (EMG):

```
RMS = sqrt( mean(signal^2) )
```

7. Archivos de salida

Archivo	Contenido
results/figures/run_001_ECG.png	Figura del pipeline ECG (72 bpm)
results/figures/run_002_ECG.png	Figura del pipeline ECG (55 bpm)
results/figures/run_003_EMG.png	Figura del pipeline EMG
results/metrics.csv	Bitácora de métricas de los 3 experimentos
results/UML_clases.png	Diagrama UML de clases del proyecto

Las carpetas se crean automáticamente en el constructor de ExperimentRunner mediante:

```
os.makedirs(self._figures_dir, exist_ok=True)
```

Las figuras incluyen una etapa por subplot: señal original, tras NotchFilter, tras BandpassFilter y detección de eventos con marcadores rojos.

8. Cómo ejecutar el proyecto

Requisitos

- Python 3.8 o superior
- Instalar dependencias:
`pip install numpy scipy matplotlib`

Ejecución

```
python main.py
```

El script ejecuta 3 experimentos por defecto: ECG a 72 bpm, ECG a 55 bpm y EMG. Al finalizar imprime las métricas en consola y guarda todos los archivos de salida en results/.

Cargar una señal real

Para reemplazar la señal sintética por datos reales desde un archivo CSV:

```
import numpy as np
from biosignal import BioSignal

datos = np.loadtxt('mi_ecg.csv', delimiter=',')
ecg = BioSignal(data=datos, fs=500.0, signal_type='ECG', name='ECG_real')
```

Nota: el CSV debe contener una sola columna de muestras 1D. Ajustar fs al valor real de adquisición.

9. Flujo de ejecución

El flujo completo al ejecutar main.py es el siguiente:

- main.py genera las señales biomédicas (sintéticas o reales) como objetos BioSignal.
- Se construyen los pipelines con add(): NotchFilter → BandpassFilter → EventDetector.
- ExperimentRunner.run() invoca pipeline.run(signal).
- Pipeline aplica cada Processor en orden, guardando señales intermedias.
- EventDetector guarda métricas en _metrics y eventos en _events.
- ExperimentRunner llama a _save_figure() y genera el PNG por etapas.
- save_metrics_csv() escribe todos los registros en results/metrics.csv.

10. Bibliografía

- Curso de matemáticas en python:
<https://www.youtube.com/playlist?list=PLgHCrivozlb11Si9m9viTdnZv5IW2ucsP>
- Uso de librerías:

- **Harris, C. R. et al.** — *Array programming with NumPy*. Nature, 585, 357–362, 2020.
Artículo oficial de numpy
- **Hunter, J. D.** — *Matplotlib: A 2D graphics environment*. Computing in Science & Engineering, 9(3), 90–95, 2007.
Artículo oficial de matplotlib
- **Documentación oficial de Scipy:**<https://docs.scipy.org/doc/scipy/reference/signal.html>
- Herramientas de apoyo: Anthropic, "Claude (claude-sonnet-4-6)," Large language model, 2026: <https://claude.ai>